Static Binding

Added in version	1.0.3
Status	Up-to date

What is static binding
Settings
Sample overview
    Code reference
How it works internally
Where to use?

What is static binding

Static binding allows to mark static void method with `[DebugPanelButtonAttribute]` and it will appear in debug panel. It supports path for buttons *(panel → tab → group... → button)* to determine where in panel's hierarchy button should be located.

Attribute **Result in UI**

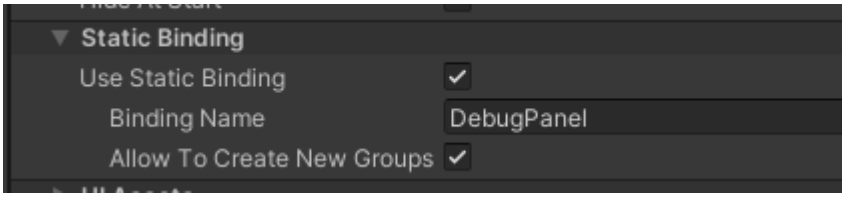


Settings

In order for static binding work corresponding option should be checked - `Use Static Binding`

`Binding Name` - you can bind to different panels, this is the identifier of this panel.

`Allow To Create New Groups` - if button attribute specifies path/hierarchy of panels that does not exist it will need to create missing hierarchy elements, otherwise buttons won't be spawned.



Sample overview

Sample contains `Player.cs` that provides health related actions, like heal/damage/kill/revive.

All of the actions is wrapped with static methods, and this methods marked with `[DebugPanelButtonAttribute]`.

Paths examples used:

- *Player/Heal/5* - Group “Player” → Group “Heal” → Button “5”
- *Player/Kill* - Group “Player” → Button “Kill”
- *1/2/3/4* - Group “1” → Group “2” → Group “3” → Button “4”

Code reference

Here is `Player.cs` script, there is also `MobFactory.cs` that also contributes to static binding


ublic class Player : MonoBehaviour
public void Heal(float amount)
{
Health += amount;
public DebugPanelButtonAttribute(string debugPanelName, string tab, string path)
in class nickeltin.DebugPanel.Runtime.Binding.DebugPanelButtonAttribute
Creates a button in specified DebugPanel => Tab => Groups[] => with Name
Params: debugPanelName — There might be multiple debug panels, so they are identified by names. If empty, will be ignored and will be added to all debug panels.
tab — There might be multiple tabs in debug panel. If empty, first tab of panel will be used.
path — Patch consists of groups separated with '/', where last element is button name. 1. "Player/Health/-10" 2. "Player/Health/+10" This will spawn two buttons (-10 and +10) in the same group.
[AttributeUsage(AttributeTargets.Method, AllowMultiple=true)]
public class DebugPanelButtonAttribute
: Attribute
in namespace nickeltin.DebugPanel.Runtime.Binding
Method marked with this attribute will appear in a debug panel if it exists, at a specified path. Method signature: - static - void - no parameters Works both in the editor and in build.
private const string DEBUG_ROOT = "Player/";
private const string HEAL = DEBUG_ROOT + "Heal/";
[DebugPanelButton(path: HEAL + "5")]
private static void Heal5() => instance.Heal(amount: 5);
[DebugPanelButton(path: HEAL + "15")]
private static void Heal15() => instance.Heal(amount: 15);
[DebugPanelButton(path: HEAL + "30")]
private static void Heal30() => instance.Heal(amount: 30);
[DebugPanelButton(path: DEBUG_ROOT + "Kill")]
private static void Kill_Debug() => instance.Kill();
[DebugPanelButton(path: DEBUG_ROOT + "Revive")]
private static void Revive_Debug() => instance.Revive();
[DebugPanelButton(path: "1/2/3/4", PreferHorizontalGroups = true)]
private static void HorizontalAlignment1() { }
[DebugPanelButton(path: "1/5/6/7")]
private static void HorizontalAlignment2() { }
[DebugPanelButton("", tab: "Secret Bindings", path: "Player/PlayerSecretOption")]
private static void PlayerSecretOption(){...}

Content provider PopulateContent just defines empty groups. If is false then only this created structure will be spawned.

```
public override void PopulateContent(DebugPanel.Runtime.DebugPanel panel)
{
    using (new Group(panel, GroupOptions.Expandable, groupName: "Player"))
    {
        using (new Group(panel, GroupOptions.Horizontal, groupName: "Heal")) { }
    }

    using (new Group(panel, GroupOptions.Expandable, groupName: "Mobs"))
    {
        using (new Group(panel, GroupOptions.Horizontal, groupName: "Spawn")) { }

        using (new Group(panel, GroupOptions.Horizontal, groupName: "Despawn")) { }
    }

    using (new TabGroup(panel, tabName: "Secret Bindings"))
    {
        using (new Group(panel, GroupOptions.Vertical, groupName: "Player")) { }
    }
}
```

How it works internally

In editor it's quite simple, you can get hold of all methods with attribute pretty fast with TypeCache.

For build it uses baking technique - caches method paths and corresponding types into serialized asset that gets force included into the build. Caching at build time allows to avoid any performance hits associated with going over all assemblies and reflections.

> 👌 So yeah, it works in both editor and build without any performance overhead since everything is cached at build time

Where to use?

For some quick prototyping it might be useful, or when you want to add support for debug panel to you system, without linking it directly to ☰ Simple Content Provider.

However its more convenient to make a custom Content Provider when your debug panel grown big. Having all the control over layout and single place to link into your game systems without necessary static context is way better.



Try to check secret player bindings)